

Attention Via Convolutional Nearest Neighbors

Mingi Kang, Jeová Farias Sales Rocha Neto
Bowdoin College
255 Maine Street, Brunswick, ME, USA
{mkang2, j.farias}@bowdoin.edu

Abstract

The shift from Convolutional Neural Networks to Transformers has reshaped computer vision, yet these two architectural families are typically viewed as fundamentally distinct. We argue that convolution and self-attention, despite their apparent differences, can be unified within a single k -nearest neighbor aggregation framework. The critical insight is that both operations are special cases of neighbor selection and aggregation; convolution selects neighbors by spatial proximity, while attention selects by feature similarity, revealing they exist on a continuous spectrum. We introduce Convolutional Nearest Neighbors (ConvNN), a unified framework that formalizes this connection. Crucially, ConvNN serves as a drop-in replacement for convolutional and attention layers, enabling systematic exploration of the intermediate spectrum between these two extremes. We validate the framework’s coherence on CIFAR-10 and CIFAR-100 classification tasks across two complementary architectures: (1) Hybrid branching in VGG improves accuracy on both CIFAR datasets by combining spatial-proximity and feature-similarity selection; and (2) ConvNN in ViT outperforms standard attention and other attention variants on both datasets. Extensive ablations on k values and architectural variants reveal that interpolating along this spectrum provides regularization benefits by balancing local and global receptive fields. Our work provides a unifying framework that dissolves the apparent distinction between convolution and attention, with implications for designing more principled and interpretable vision architectures.

1. Introduction

Deep learning architectures for computer vision have undergone a fundamental transformation over the past two decades. Convolutional neural networks (CNNs) dominated through pioneering architectures like AlexNet [19], VGG [27], and ResNet [14]. More recently, Vision Transformers (ViT) [10] have reshaped the landscape with attention-based models achieving state-of-the-art performance across numerous

vision tasks. Yet despite these successes, convolution and attention are typically treated as fundamentally distinct mechanisms suited for different situations.

Traditionally, convolution operates through local spatial aggregation, where each output is computed from a fixed spatial neighborhood. In contrast, self-attention [29] enables global feature aggregation by computing similarity-based weighted combinations across all spatial positions. However, beneath these surface differences lies a common principle: both operations aggregate features from selected neighbors. The distinction is in neighbor selection—convolution selects by spatial proximity, while attention selects by learned feature similarity.

We propose *Convolutional Nearest Neighbors (ConvNN)*, a unified framework that formalizes the interconnection between convolution and attention through k -nearest neighbor (k -NN) selection. Our key insight is that both operations are specialized instances of neighbor aggregation, differing in their selection strategy. Crucially, ConvNN can be exactly configured to recover standard convolution and exactly recover standard attention, proving both are points on a continuous spectrum.

The flexibility of ConvNN enables systematic exploration of this spectrum. Rather than choosing between convolution or attention, we can design hybrid configurations that combine spatial and feature-based neighbor selection. Our experiments on CIFAR-10 and CIFAR-100 demonstrate that these configurations achieve consistent improvements in accuracy when integrated into both CNN (VGG) and Transformer (ViT) architectures. Beyond performance gains, hybrid configurations provide interpretability by explicitly balancing local spatial features with global feature dependencies. This unified framework provides a foundation for designing next-generation operations and architectures that leverage the strengths of both mechanisms.

2. Related Works

Convolution and Attention in Vision Convolution has dominated computer vision through architectures like AlexNet [19], VGG [27], and ResNet [14], providing ef-

efficient local feature extraction with spatial inductive biases. Recently, Vision Transformers (ViT) [10] and other attention-based models [20, 21] have achieved comparable or superior performance through self-attention [29], which captures long-range dependencies without spatial constraints, but at a larger computational requirement.

Several variants of the self-attention mechanism have been proposed to reduce its quadratic computational and memory complexity. *Local Attention* [1] restricts each query to attend only to nearby tokens within a fixed window, thereby exploiting locality while lowering the attention cost from $O(n^2)$ to $O(nw)$, where w is the window size. *Sparse Attention* [6], further generalizes this idea by enforcing structured sparsity patterns in the attention matrix, allowing each token to attend to a predefined subset of positions. These approaches preserve most of the modeling capacity of global attention while making Transformer architectures more scalable for long sequences.

Connections Between Convolution and Attention Recent work began exploring the theoretical relationships between these seemingly distinct operations. Cordonnier et al. [8] provided evidence that self-attention with relative positional encoding can express any convolutional operation, suggesting convolution and attention may exist on a spectrum. *Non-Local Neural Networks* [31] and channel-based attention mechanisms [16] introduced global aggregation mechanisms, demonstrating that operations beyond local convolution can enhance feature learning. Furthermore, recent architectures have explicitly combined both operations. *Attention Augmented Convolutional Networks* [2] augment convolutional layers with self-attention. *CoAtNet* [32] systematically studies depthwise convolutions followed by attention blocks, showing that this combination achieves strong performance. *ConViT* [9] introduces gated positional self-attention that gradually transitions from convolutional-like to attention-like behavior. *Neighborhood Attention* [13] employs sliding windows to reduce attention’s computational cost while preserving spatial locality. While these hybrid architectures demonstrate the benefits of combining local and global operations, they treat convolution and attention as distinct operations combined through architectural composition rather than unified through a common theoretical principle.

k -Nearest Neighbors in Deep Learning The k -nearest neighbor (k -NN) principle has recently reemerged as a valuable design component in neural network architectures. *Neural Nearest Neighbors Networks* [23] first explored the integration of k -NN reasoning primarily for few-shot learning. More recently, k -NN mechanisms have been incorporated into Transformers to mitigate the quadratic computational and memory costs of self-attention. In particular, *k -NN Attention for Boosting Vision Transformers (KVT)* [30] enhances

Vision Transformers by introducing a k -nearest-neighbor attention mechanism that restricts each token’s attention to its most relevant local neighbors rather than the entire sequence. Formally, KVT introduces a k -NN masking function that constrains each query to attend only to its k most similar keys, according to the traditional scaled dot-product similarity. The selected neighbor values are then normalized via a softmax operation, and the overall attention is computed with a reduced complexity of $O(nk \log k)$ —lower than the traditional $O(n^2d)$ of full self-attention—where n denotes the number of tokens and d the embedding dimension. By adaptively selecting the k most similar keys for each query in feature space, KVT effectively balances local and global context modeling, leading to better performance and faster convergence compared to standard ViTs.

Similarly, *Unlimiformer* [3] addresses attention computation complexity by using k -NN indices to compute only k -NN distances. *Routing Transformers* [25] rely on k -means clustering to model sparse attention matrices; this allows each token to attend to its k closest tokens. Furthermore, in *k -NN Attention Demystified* [12], a theoretical framework for k -NN attention is presented, reframing self-attention as expectations and leveraging lazy Gumbel sampling for efficient approximation. *Memory-efficient Transformers* [11] also addresses this complexity through a highly accurate approximation for vanilla attention with top- k scores in chunks of queries. These works demonstrate the value of neighbor selection as a design principle, though they treat k -NN as an enhancement to existing architectures rather than a unifying framework for understanding different operations.

3. Background

3.1. Notation

Scalar variables, vectors, and matrices, are denoted in plain lowercase (e.g., x), bold lowercase (e.g., $\mathbf{x}$), and in plain uppercase letters (e.g. X), respectively. High-order tensors are denoted by bold uppercase letters (e.g., $\mathbf{X}$). The shape of a tensor is written in square brackets (e.g., the shape $\mathbf{X} \in \mathbb{R}^{a \times b \times c}$ is $[a, b, c]$). A vector of ones in $\mathbb{R}^k$ is denoted as $\mathbf{1}_k$.

We adopt Python-style indexing notation, where square brackets (e.g., $X[i, :]$) indicate row and column selection. Given a matrix $A \in \mathbb{R}^{n \times m}$, $A[i, :]$ selects the entire i -th row of A . Given a vector mask $\mathbf{m} \in \{0, 1\}^n$, $A[\mathbf{m}, :]$ selects the rows i where $\mathbf{m}_i = 0$. A feature vector (or simply feature) in a 2D tensor is a transversal tube of unitary spatial dimensions across channels.

A vector $\mathbf{x}$ is ℓ_2 -normalized when $\|\mathbf{x}\|_2 = 1$, where $\|\cdot\|_2$ denotes the ℓ_2 norm. The operator $\text{softmax}(\mathbf{x})$ normalizes $\mathbf{x}$ into a probability distribution, with $\text{softmax}(\mathbf{x})_i = \exp(x_i) / \sum_j \exp(x_j)$. Finally, $\text{diag}(\mathbf{v})$ denotes the diagonal matrix with the entries of $\mathbf{v}$ on its main diagonal.

3.2. k -Nearest Neighbors

Let $X \in \mathbb{R}^{n \times d}$ be a matrix with columns $X = \{\mathbf{x}_i\}_{i=1}^n$, where each $\mathbf{x}_i$ is ℓ_2 -normalized. In this setting the squared Euclidean distance between two points $\mathbf{x}_i$ and $\mathbf{x}_j$ is given by $2(1 - \mathbf{x}_i^\top \mathbf{x}_j)$ and their cosine similarity is $\mathbf{x}_i^\top \mathbf{x}_j$, which reveals that, for normalized vectors, Euclidean distance is inversely related to cosine similarity.

We define k -max : $\mathbb{R}^{n \times m} \rightarrow \mathbb{R}^{n \times k}$ as the operator that returns, for each row of a matrix, the k largest values. Correspondingly, k -argmax : $\mathbb{R}^{n \times m} \rightarrow \{0, 1\}^{n \times m}$ returns a multi-hot encoding¹ indicating the positions of these top- k values. Let $\mathcal{N}_k(\mathbf{x}_i) \subset X$ be the set of k -nearest neighbors of $\mathbf{x}_i \in X$. We can compute $\mathcal{N}_k(\mathbf{x}_i)$ as a submatrix of X through $X[I_i, :] \in \mathbb{R}^{k \times d}$, where indices I are given by:

$$I = k\text{-argmax}(XX^\top) \in \mathbb{R}^{n \times n}, \quad (1)$$

and $I_i = I[i, :]$. In other words, $X[I_i, :]$, selects the rows of X corresponding to the k -nearest neighbors of $\mathbf{x}_i$ based its similarity to the other points.

3.3. Convolution and Attention

3.3.1. Convolution

Let $Z \in \mathbb{R}^{n \times c}$ be a 1D tensor with c channels and $W \in \mathbb{R}^{k \times c}$ a convolutional kernel with bias $b \in \mathbb{R}$. A 1D convolution (Conv1D) slides W across Z , computing local dot products and adding b , to produce $Y \in \mathbb{R}^{n' \times c'}$, where n' depends on the kernel size, stride, and padding. In 2D (Conv2D), the input $Z \in \mathbb{R}^{w \times h \times c}$ is convolved with kernels $W \in \mathbb{R}^{k \times k \times c}$ across both spatial dimensions, yielding $Y \in \mathbb{R}^{w' \times h' \times c'}$. For $k = 1$, the operation reduces to a pointwise linear projection across channels (a 1×1 convolution).

In *depthwise convolution* [7], instead of learning kernels that span all input channels, we assign one kernel per channel. That is, each channel $Z_i \in \mathbb{R}^{n \times 1}$ is convolved with its own kernel $W_i \in \mathbb{R}^{k \times 1}$, yielding $Y_i \in \mathbb{R}^{n' \times 1}$. These c independent convolutions capture spatial correlations within each channel separately, without mixing information across channels.

3.3.2. Attention

Given query, key, and value matrices $Q \in \mathbb{R}^{n \times h}$, $K \in \mathbb{R}^{m \times h}$, and $V \in \mathbb{R}^{m \times v}$, respectively, the dot-product attention [29] computes

$$A = \text{softmax}(QK^\top) \in \mathbb{R}^{n \times m}, \quad (2)$$

and produces the output $AV \in \mathbb{R}^{n \times v}$. We omit the usual scaling factor $\sqrt{h}$ for simplicity, as it does not affect our proposed formulation in the following.

¹A multi-hot encoded vector is a binary vector $\mathbf{v} \in \{0, 1\}^n$ such that $\mathbf{v}_i = 1$ if a given condition is true for index i , and 0 otherwise.

4. Convolutional Nearest Neighbors

4.1. Core Framework

We now present ConvNN, an operation that generalizes both convolution and attention, is built upon three core steps: *similarity computation*, *neighbor selection and modulation*, and *weighted aggregation*. Let $X \in \mathbb{R}^{n \times c}$ denote the input feature matrix. ConvNN operates as follows:

1. Similarity computation:

Let $f_Q : \mathbb{R}^{n \times c} \rightarrow \mathbb{R}^{n \times h}$, $f_K : \mathbb{R}^{n \times c} \rightarrow \mathbb{R}^{n \times h}$, and be learnable linear projections. Let $Q = f_Q(X)$ and $K = f_K(X)$. After applying ℓ_2 normalization to the rows of Q and K , feature similarities are computed in $\mathbb{R}^h$ rather than $\mathbb{R}^c$, and the neighbors of each feature $\mathbf{x}_i$ are extracted from V using these indices of these projected representations. ConvNN employs a cosine similarity between these normalized features, which means that:

$$S = QK^\top. \quad (3)$$

2. Neighbor selection and modulation:

Let $f_V : \mathbb{R}^{n \times c} \rightarrow \mathbb{R}^{n \times v}$ be another learnable linear projection, and define $V = f_V(X)$. For each feature vector $\mathbf{x}_i \in X$, let $\mathcal{N}_k^{\text{proj}}(\mathbf{x}_i)$ denote the set of k rows of V corresponding to the k rows of K most similar to $Q[i, :]$. We refer to the elements of $\mathcal{N}_k^{\text{proj}}(\mathbf{x}_i)$ as the *projected neighbors* of $\mathbf{x}_i$. The similarity values and indices of these neighbors are respectively denoted as:

$$\mathbf{s}_i = k\text{-max}_k(S)[i, :], \quad (4)$$

$$I_i = k\text{-argmax}_k(S)[i, :]. \quad (5)$$

We introduce a placeholder function $\rho : \mathbb{R}^k \rightarrow \mathbb{R}^k$ that defines the weighting scheme, instantiated either as a constant vector ($\rho(\mathbf{z}) = \mathbf{1}_k$) or a softmax normalization ($\rho(\mathbf{z}) = \text{softmax}(\mathbf{z})$). Let $S_i = \text{diag}(\rho(\mathbf{s}_i))$ denote the resulting diagonal weighting matrix.

ConvNN then constructs, for each $\mathbf{x}_i$, a *neighborhood matrix* $X_{\text{nn},i}$ defined as:

$$X_{\text{nn},i} = S_i V[I_i, :] \in \mathbb{R}^{k \times c}, \quad (6)$$

which represents the k projected neighbors of $\mathbf{x}_i$, each scaled by the corresponding weight in $\rho(\mathbf{s}_i)$ and ordered by similarity.

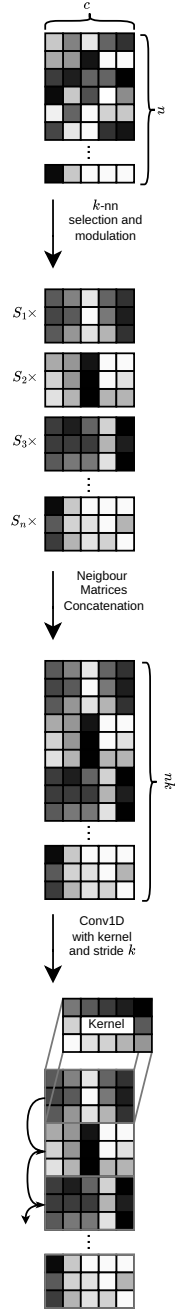


Figure 1. The ConvNN operation

4. Weighted aggregation:

Let $X_{nn} \in \mathbb{R}^{(kn) \times c}$ denote the concatenation of all neighborhood matrices $X_{nn,i}$ along the spatial dimension. CONVNN applies a CONV1D operation (either standard or depthwise) with c' kernels, stride k , and kernel size k to X_{nn} , yielding the output $Y \in \mathbb{R}^{c' \times n}$.

For input tensors with spatial structure, such as those of shape $[c, w, h]$, CONVNN first flattens the spatial dimensions to obtain a tensor of shape $[wh, c]$, applies the procedure described above, and then reshapes the result back to $[c', w, h]$. An overview of the CONVNN is illustrated in Figure 1, and its official PyTorch [24] code is detailed in Algorithm 1.

Algorithm 1. PyTorch code of our ConvNN module.

```
class ConvNN(nn.Module):
    def __init__(self, k=9, rho='softmax'):
        super().__init__()
        self.k, self.rho = k, rho
        self.conv1d = nn.Conv1d(kernel_size=k, stride=k)
        self.w_k = nn.Conv1d(kernel_size=1, stride=1)
        self.w_q = nn.Conv1d(kernel_size=1, stride=1)
        self.w_v = nn.Conv1d(kernel_size=1, stride=1)

    def forward(self, x):
        # Project the input
        k, q, v = self.w_k(x), self.w_q(x), self.w_v(x)

        # Compute Similarities
        k_norm = F.normalize(k, p=2, dim=1)
        q_norm = F.normalize(q, p=2, dim=1)
        S = k_norm.T @ q_norm

        # Select Neighbors
        kmax, kargmax = torch.topk(S, k=self.k, dim=-1)
        if self.rho == 'softmax':
            kmax = torch.softmax(kmax, dim=-1)

        # Neighbor Gathering and Modulation
        v = v.unsqueeze(-1).expand(*x.shape, self.k)
        kmax = kmax.unsqueeze(1)
            .expand(*x.shape, self.k)
        kargmax = kargmax.unsqueeze(1)
            .expand(*x.shape, self.k)

        x_nn = kmax * torch.gather(v, dim=2,
                                   index=kargmax)

        # Neighbor Aggregation
        b, c, _ = x.shape
        return self.conv1d(x_nn.view(b, c, -1))
```

4.2. Extensions

To enhance the expressiveness of CONVNN, we introduce two extensions that enable it to interpolate between purely feature-based and spatially-aware neighbor selection.

4.2.1. Positional Encoding

To incorporate spatial structure when beneficial, CONVNN optionally concatenates positional coordinates to the input along the channel dimension. For a 1D input $[c, l]$, a single channel increasing linearly with position is added, while for a 2D input $[c, w, h]$, two channels encoding horizontal and vertical coordinates (i, j) are appended. The resulting tensors have shapes $[c + 1, l]$ and $[c + 2, w, h]$, respectively. This positional encoding enables CONVNN to blend feature-based and spatially aware similarity computations.

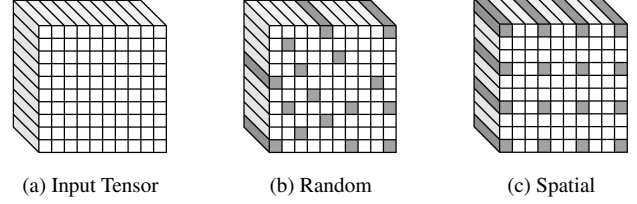


Figure 2. Neighbor candidate selection strategies for $r = 16$. Candidates features are highlighted in gray.

4.2.2. Sparse Neighbor Search

To reduce the computational burden of k -NN search from $O(n^2 \log n)$ to $O(nr \log r)$, we restrict the search to a set of $r \ll n$ candidate neighbors. We select these candidates using one of two strategies:

- **Random Selection** (see Fig. 2b): We randomly sample r features from X to form X^{sparse} , i.e., $X^{\text{sparse}} = X[I_r, :]$ where I_r contains r indices drawn uniformly without replacement. This strategy preserves feature diversity while maintaining a reduced search space.
- **Spatial Selection** (see Fig. 2c): For a 1D tensor $\mathbf{X} \in \mathbb{R}^{c \times l}$ and a subsampling rate r , we extract a reduced tensor $\mathbf{X}^{\text{sparse}} \in \mathbb{R}^{c \times \frac{l}{r}}$ such that $\mathbf{X}^{\text{sparse}}[:, i] = \mathbf{X}[:, ri]$, effectively selecting features at regular intervals along the sequence. Analogously, for a 2D tensor $\mathbf{X} \in \mathbb{R}^{c \times w \times h}$ and $r' = \sqrt{r}$, we extract a subtensor $\mathbf{X}^{\text{sparse}} \in \mathbb{R}^{c \times \frac{w}{r'} \times \frac{h}{r'}}$ such that $\mathbf{X}^{\text{sparse}}[:, i, j] = \mathbf{X}[:, r'i, r'j]$, thereby subsampling features at uniform spatial intervals.

Beyond computational efficiency, this technique reduces locality bias where the closest feature vectors tend to be spatially adjacent, forcing CONVNN to learn potentially beneficial long-range relationships [17].

4.2.3. Hybrid Branching Layer

A key distinction between CONVNN and standard convolution lies in their receptive behavior: while convolutions aggregate information locally from spatially adjacent features, CONVNN captures long-range dependencies, allowing global context to influence each representation.

Building upon prior work that integrates local feature extraction with global feature modeling [4, 22, 28], we introduce a hybrid *branching layer* that combines convolutional and CONVNN components. We first set the CONVNN projectors as identity functions. Then this layer processes the input through two parallel branches—one convolutional and one CONVNN—and fuses their outputs via a 1×1 convolution. A parameter $\lambda \in [0, 1]$ controls the proportion of output channels allocated to each branch, ensuring a balanced contribution between local and global processing. Both branches use the same kernel size (or neighbor count), keeping the parameter budget comparable while regulating their relative influence through λ .

5. Connection to Prior Literature

5.1. Traditional Convolution

Traditional convolution emerges as a special case of CONVNN where neighbors are selected by spatial proximity rather than feature similarity. This can be achieved by using positional encoding, choosing projections f_K and f_Q that filter all channels except spatial dimensions, setting f_V as the identity function and choosing $\rho = \mathbf{1}_k$ (constant weighting). In this configuration, the k -nearest neighbors of a feature are necessarily the spatially adjacent features, reducing CONVNN to standard convolution with receptive field size given by k .

5.2. Attention Mechanism

Next, we show that the attention mechanism in Eq. 2 can be recovered as a special case of CONVNN by setting $k = n$, $\rho = \text{softmax}$, and applying a depthwise convolution with unit weights and zero bias during the weighted aggregation phase. Consider first the multiplication AV in Eq. 2. The i -th row of this product can be viewed as the sum of the rows of V , each scaled by the corresponding attention weight in $A[i, :] = \text{softmax}(Q[i, :]K^\top)$.

Under the above settings, CONVNN reproduces this computation. In Eq. 6, the modulation step scales the same rows of V , now arranged as $V[I_i^{\text{proj}}, :]$, by the values in $\rho(\mathbf{s}_i) = \text{softmax}(Q[i, :]K^\top)$. Applying a bias-free depthwise convolution with constant weights equal to one effectively performs the same summation across the scaled rows of V . When applied to all i , this process yields exactly AV . Furthermore, relaxing the condition $k = n$ naturally recovers the localized attention formulation proposed in KVT [30].

6. Experimental Evaluation

We evaluate the performance of CONVNN against convolutional and attention-based variants on the CIFAR-10 and CIFAR-100 classification benchmarks [18]. These datasets provide a controlled setting for assessing theoretical frameworks and isolating the contributions of the unified CONVNN formulation. CIFAR-10 and CIFAR-100 remain standard benchmarks for controlled architectural studies in computer vision [15] and have also been recently adopted in the evaluation of transformer-based architectures [5].

For convolution-based experiments, we employ a VGG convolutional neural network (VGG-11) [27] architecture. For attention-based experiments, we use Vision Transformer (in its ViT-Tiny version) [10] with 12 layers, 192 hidden dimensions, 768 MLP dimensions, attention dropout and MLP dropout to 0.1. For simplicity, we decided to set the number of heads in our attention-based models to 1. In each case, when not stated otherwise, we replace all their individual convolution or attention operations by CONVNN modules when comparing both paradigms.

We trained all models using AdamW optimizer with a learning rate of 10^{-4} and weight decay of 0.01 for 150 epochs with a batch size of 256. We apply gradient clipping with a maximum norm of 1.0 and use cross-entropy loss as the training criterion. We do not employ a learning rate scheduler to maintain consistency and reproducibility across experiments. All experiments are implemented in PyTorch 2.0 and executed on NVIDIA A100 PCIe 80 GB GPUs.

6.1. CONVNN vs. Traditional Convolution

In this section, for experimental simplicity and to facilitate direct comparison with standard convolutions, we set the CONVNN projectors to identity functions and fix $\rho(\mathbf{z}) = \mathbf{1}_k$. Only the VGG-11 architecture is evaluated in this setting.

Table 1 shows the performance of our hybrid CONVNN branching modules (see Sec. 4.2.3) for varying values of λ . In these experiments, $k = 9$ is used for CONVNN, and 3×3 kernels are used in the CONV2D layers. Sparse neighbor

Table 1. Test loss and accuracy for VGG-11 trained on CIFAR-10 and 100 when varying λ in your hybrid branching module. Bold values indicate values that outperform those of the CONV2D baseline.

Without Positional Encoding					
Layer Type	CIFAR-10		CIFAR-100		GFLOPS
	Test Loss	Acc (%)	Test Loss	Acc (%)	
CONV2D	1.529	80.65	3.939	44.83	0.293
λ	0.125	1.393	81.10	3.905	45.08
	0.250	1.467	81.21	3.518	47.87
	0.375	1.422	80.95	3.578	48.34
	0.500	1.424	81.26	3.474	48.37
	0.625	1.403	80.79	3.281	49.74
	0.750	1.479	79.58	3.516	46.93
	0.875	1.332	78.18	3.906	45.72
CONVNN	2.694	54.00%	5.011	29.50%	0.325
# Params.	130.015M		130.384M		
With Positional Encoding					
Layer Type	CIFAR-10		CIFAR-100		GFLOPS
	Test Loss	Acc (%)	Test Loss	Acc (%)	
CONV2D	1.529	80.65	3.939	44.83	0.293
λ	0.125	1.420	80.97	3.665	47.73
	0.250	1.447	80.91	3.702	47.65
	0.375	1.386	82.08	3.696	48.39
	0.500	1.213	81.77	3.588	48.80
	0.625	1.341	80.81	3.323	49.23
	0.750	1.283	80.26	3.533	48.78
	0.875	1.225	78.70	3.539	46.26
CONVNN	2.388	57.15	4.586	32.33	0.331
# Params.	130.015M		130.384M		

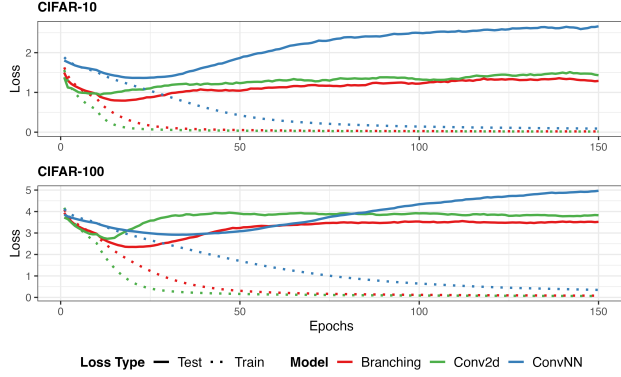


Figure 3. Loss comparison on VGG-11 training. In branching, we set $\lambda = 0.5$. For visualization purposes, all loss curves are smoothed using a moving average with a window size of 10.

search is disabled for clarity.

Table 1 reports results when raw features are used to compute neighbor similarities, or when positional encodings are added before similarity computation (see Sec. 4.2.1). Despite the pure ConvNN configuration ($\lambda = 1$) underperforming relative to the pure convolutional case ($\lambda = 0$), the hybrid branching model consistently surpasses both across most of the range $0 < \lambda < 1$, without introducing additional learnable parameters. This effect is particularly pronounced on CIFAR-100, where the branching model improves accuracy by nearly 5%. The inclusion of positional encodings yields further, albeit modest, gains across both datasets, but is particularly noticeable in pure ConvNN.

Figure 3 presents the training and test loss trajectories for VGG-11 on CIFAR-10 and CIFAR-100 using Conv2D, ConvNN, and the branching variant with $\lambda = 0.5$, where similarities are computed from raw input features. These curves highlight the advantages of the hybrid approach: while Conv2D

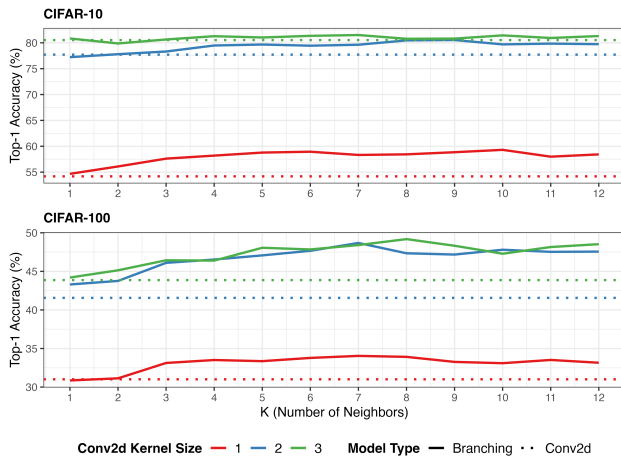


Figure 4. Impact of k to ConvNN's performance on VGG-11 training against the Conv2D baseline.

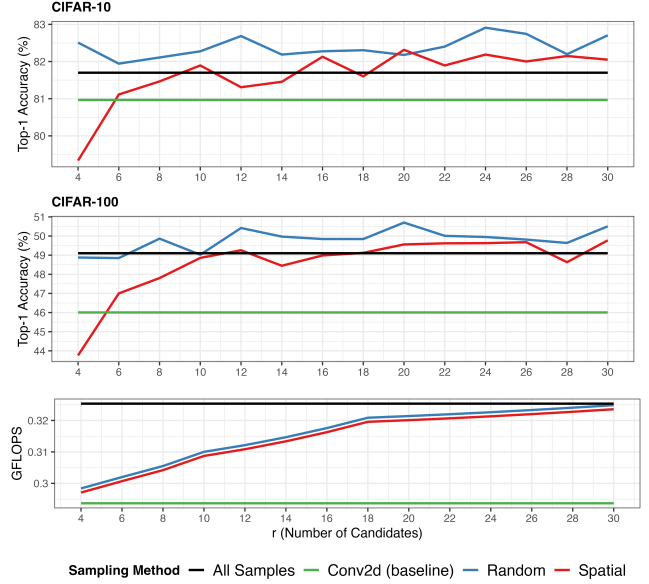


Figure 5. Impact of r to ConvNN's performance on VGG-11 training. The Random and Spatial candidate selection curves are slightly offset vertically for clarity, had the same runtime throughout the experiments.

quickly overfits, the branching configuration exhibits greater resistance to overfitting, achieves lower overall loss (around epoch 40), and converges to the best final performance by a good margin. Interestingly, even though its representational capacity is split between Conv2D and ConvNN, the branching model outperforms both, underscoring the value of jointly balancing local and global feature learning.

Figure 4 considers the branching module ($\lambda = 0.5$) performances for varying a varying number of neighbors (k) and kernel sizes on its ConvNN and Conv2D branches, respectively. Three important observations can be made from these results: (1) the addition of a ConvNN branch consistently improves the otherwise sole Conv2D performances, (2) the choice of a higher k in most cases does usually not reflect a proportionally higher performances, which means that the same accuracy increments can be achieved at a lower learnable parameter need, and (3) ConvNN's contributions are more evident at CIFAR-100, suggesting that it may be more suitable to datasets with a lower datapoint per class ratio.

Finally, Figure 5 presents a quantitative comparison of the sparse neighbor search strategies described in Sec. 4.2.2 against both the baseline ConvNN (which considers all features as neighbor candidates) and the standard Conv2D. As expected, the sparse variants significantly reduce ConvNN's runtime, approaching that of Conv2D for smaller values of r . Moreover, increasing the number of candidate neighbors generally improves the overall performance of the proposed module, and in some cases even surpasses the configuration that uses all features. This behavior is intuitive: by enforcing

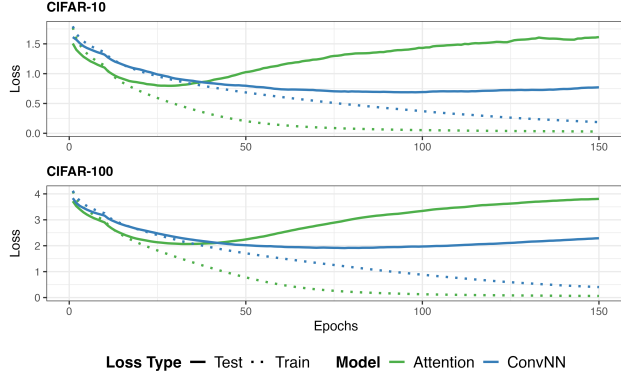


Figure 6. Loss comparison on ViT-Tiny training. CONVNN with Random Candidate Selection sampling $r = 32$. For visualization purposes, all loss curves are smoothed using a moving average with a window size of 10. Learning rate was set as 10^{-4} .

neighbor selection over a spatially constrained subset, CONVNN is encouraged to capture more diverse and spatially distant relationships, thereby mitigating locality bias and complementing the locality-focused CONV2D branch within the hybrid architecture. Interestingly, the random strategy consistently outperforms both the spatial and all-feature approaches, even for small r . Combined with its lower computational cost, this makes it an appealing alternative to CONV2D.

6.2. CONVNN vs. Attention

In this section, we present quantitative comparisons between the standard attention mechanism and our proposed CONVNN variant. To ensure a fair comparison, we employ the full set of linear projectors, omit positional encoding, and set $\rho = \text{softmax}$. Following its connection to KVT (see

Sec. 5.2), CONVNN uses a 1D depthwise convolution with weights initialized uniformly to 1 and no bias. Unless otherwise stated, we set $k = 9$ and $r = 32$. We further evaluate CONVNN against several representative attention variants—the aforementioned KVT Attention [30], Local Attention [1], and Sparse Attention [6]—in addition to the original self-attention formulation [29]. All methods are implemented and trained from scratch within the Tiny-ViT framework [10].

Table 2 compares the performance of CONVNN against the various attention mechanisms evaluated. While CONVNN underperforms when using either all features as potential neighbors or only uniformly spatially sampled candidates, it achieves substantially lower losses and faster runtimes when our random selection is employed. These gains are obtained with only a small increase in the overall parameter count. Figure 6 further contributes to this picture as it shows how CONVNN avoids overfitting to the data, while that is observed in the traditional attention methodology in early epochs.

Figure 7 shows the impact of k on the performance of our CONVNN-based attention module compared to both the baseline attention mechanism and KVT. When all features are used as potential neighbors, CONVNN achieves performance comparable to KVT. However, when employing our random neighbor selection strategy, CONVNN substantially outperforms both KVT and standard attention. As with previous observations, increasing k beyond a certain value yields only marginal accuracy gains, indicating that CONVNN effectively captures the most relevant neighbors even with smaller k .

Figure 8 compares the random and spatial candidate selection strategies against both the baseline attention mechanism [29] and the CONVNN variant using all features as potential neighbors. Across all configurations, CONVNN demonstrates faster runtimes than standard attention. With $r = 16$, both sparse neighbor search strategies achieve runtimes approximately 20% faster than the baseline. In terms of classification accuracy, we observe that the spatial candidate selection performs consistently below the baseline, whereas the random selection strategy outperforms it in all scenarios—including those with the smallest candidate pools, where it also runs the fastest. These results further highlight the advantage of encouraging CONVNN to capture long-range relationships through randomized neighbor selection. This effect is less pronounced in the spatial sampling scheme, which still tends to emphasize local neighborhoods and may therefore overlook distant but informative features.

7. Discussion

Our experiments reveal both the promise and limitations of unifying convolution and attention through k -NN. The CONVNN framework successfully demonstrates that these operations exist along a spectrum of neighbor selection strategies, with convolution representing fixed spatial selection and attention representing global, feature-based selection.

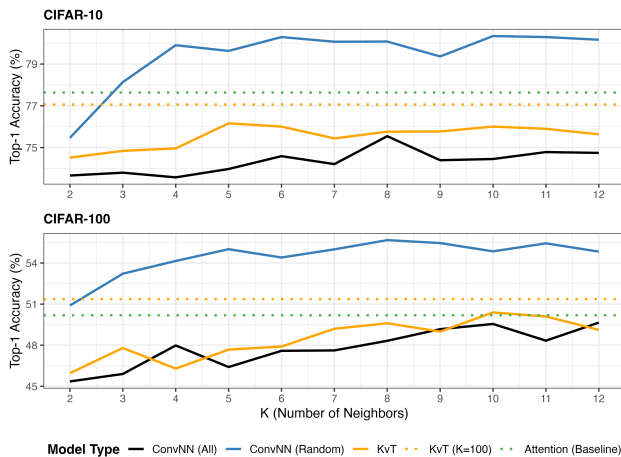


Figure 7. Impact of k to CONVNN's performance on ViT-Tiny training against both original Attention and KVT baselines. The KVT model is also evaluated across varying k , and at its best performance at $k = 100$.

Table 2. Test loss and accuracy for ViT-Tiny on CIFAR datasets. Bold values indicate best results; underline indicate second best.

Layer Type	CIFAR-10				CIFAR-100			
	Test Loss	Acc (%)	GFLOPS	# Params.	Test Loss	Acc (%)	GFLOPS	# Params.
Attention [29]	<u>1.558</u>	<u>77.41%</u>	2.507	<u>5.479M</u>	3.817	50.07%	2.507	<u>5.496M</u>
Local-Attention [1]	1.626	76.51%	1.991	4.299M	<u>3.688</u>	49.73%	1.991	4.317M
KVT-Attention ($k = 9$) [30]	1.775	75.77%	2.508	<u>5.479M</u>	3.883	48.21%	2.508	<u>5.496M</u>
KVT-Attention ($k = 100$) [30]	1.593	76.41%	2.508	<u>5.479M</u>	3.738	<u>50.34%</u>	2.508	<u>5.496M</u>
Sparse-Attention [6]	2.026	72.50%	2.509	<u>5.479M</u>	4.565	44.62%	2.509	<u>5.496M</u>
CONVNN (All features)	1.978	73.67%	2.333	5.500M	3.983	48.67%	2.333	5.517M
CONVNN (Random Selection)	0.779	79.82%	<u>2.037</u>	5.500M	2.341	53.60%	<u>2.037</u>	5.517M
CONVNN (Spatial Selection)	2.737	63.01%	<u>2.037</u>	5.500M	5.020	38.22%	<u>2.037</u>	5.517M

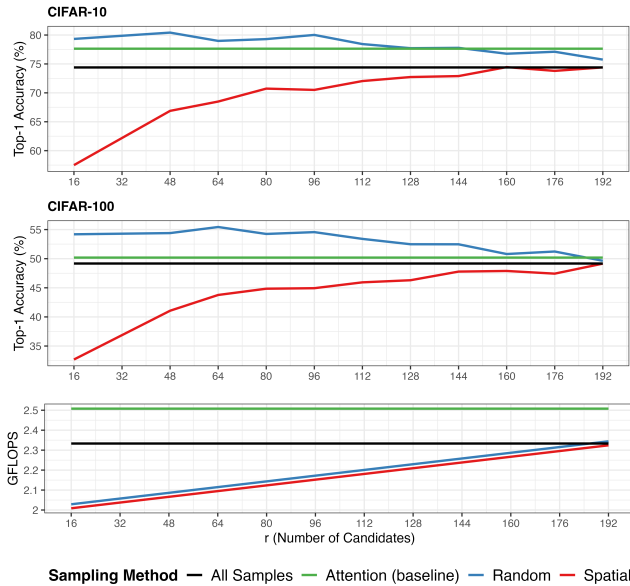


Figure 8. Impact of r to CONVNN’s performance on ViT-Tiny training. The Random and Spatial candidate selection curves are slightly offset vertically for clarity, had the same runtime throughout the experiments.

A key finding is the strong performance of random selection in CONVNN compared to considering all samples. This highlights that when employing a global receptive field, selecting the closest neighbors based on learned feature is more effective than arbitrary spatial or attention-based patterns. Notably, in the CONVNN-Attention variant, which fully utilizes its linear projectors and softmax normalization, random selection outperforms all other attention baselines, suggesting that attending to all positions may be unnecessary for learning global dependencies. This insight challenges the assumption underlying standard attention that all positions should influence the output.

Ablation studies on k values reveal an important distinction between convolution and attention. While larger convolutional kernels consistently improve performance (e.g.,

kernel size $1 \rightarrow 2$ or 3), the gains from increasing k in CONVNN are modest. Interestingly, in ViT-Tiny experiments, CONVNN using all features underperforms KVT, whereas the random selection variant outperforms all attention baselines. This indicates that CONVNN’s advantage arises from flexible neighbor selection rather than the number of neighbors considered.

We also observe significant regularization effects, particularly in convolutional architectures. Adding global receptive fields through CONVNN acts as an implicit regularizer, forcing the network to balance spatial and feature-based aggregation. This prevents over-reliance on either mechanism alone, leading to more robust feature learning.

Our work also highlights several limitations. The hard selection mechanism in CONVNN limits gradient flow and may cause training instability in certain settings. Additionally, the computational cost of k -NN search remains higher than standard convolution, though lower than full self-attention. Extension to tasks requiring dense predictions (e.g., segmentation) will require careful consideration of how to maintain spatial correspondence during neighbor selection.

8. Conclusion

We introduced *Convolutional Nearest Neighbors* (CONVNN), a unified framework bridging convolution and attention through k -nearest neighbor selection. By showing that both operations can be viewed as forms of neighborhood aggregation—based respectively on spatial proximity and feature similarity—CONVNN offers new theoretical insight into the connection between CNNs and attention-based architectures. Experiments on CIFAR-10 and CIFAR-100 demonstrate its flexibility and strong empirical performance, achieving competitive accuracy and runtime relative to both convolutional and attention baselines. Future work will leverage the CONVNN framework to systematically analyze how different architectural components of CNNs and Transformers contribute to their representational strengths, as well as to design intermediate models that extend our branching strategy to more general convolution–attention hybrids.

9. Acknowledgments

This work was supported by Bowdoin College’s Christenfeld Summer Research Fellowship 2024, John L. Roberts Fund Fall Research Grant 2025, Last Mile Education’s Google AI 2024 award and NYC STEM Award.

We are grateful to Stephen Houser and Dj Merrill for their technical assistance with Bowdoin College’s HPC system and NVIDIA GPUs, and to the Bowdoin College Computer Science department for their sustained support throughout this project.

Code is available at the following repositories:

- github.com/mingikang31/ConvNN
- github.com/mingikang31/ConvNN-Attention

References

- [1] Ignacio Aguilera-Martos, Andrés Herrera-Poyatos, Julián Luengo, and Francisco Herrera. Local attention mechanism: Boosting the transformer architecture for long-sequence time series forecasting. *arXiv preprint arXiv:2410.03805*, 2024. 2, 7, 8
- [2] Irwan Bello, Barret Zoph, Quoc Le, Ashish Vaswani, and Jonathon Shlens. Attention augmented convolutional networks. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 3285–3294. IEEE, 2019. 2
- [3] Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew Gormley. Unlimformer: Long-range transformers with unlimited length input. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. 2
- [4] Chun-Fu Chen, Rameswar Panda, and Quanfu Fan. Regionvit: Regional-to-local attention for vision transformers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. Poster. 4
- [5] Xiangyu Chen, Qinghao Hu, Kaidong Li, Cuncong Zhong, and Guanghui Wang. Accumulated trivial attention matters in vision transformers on small datasets. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3984–3992, 2023. 5
- [6] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. 2, 7, 8
- [7] François Chollet. Xception: Deep learning with depthwise separable convolutions, 2017. 3
- [8] Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. On the relationship between self-attention and convolutional layers. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. 2
- [9] Stéphane d’Ascoli, Hugo Touvron, Matthew L. Leavitt, Ari S. Morcos, Giulio Biroli, and Levent Sagun. Convit: Improving vision transformers with soft convolutional inductive biases. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pages 2286–2296. PMLR, 2021. 2
- [10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. 1, 2, 5, 7
- [11] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*. Association for Computational Linguistics, 2021. 2
- [12] Themistoklis Haris. knn attention demystified: A theoretical exploration for scalable transformers. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. 2
- [13] Ali Hassani, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. Neighborhood attention transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pages 6185–6194. IEEE, 2023. 2
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. 5
- [16] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. Squeeze-and-excitation networks. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(8):2011–2023, 2020. 2
- [17] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. 4
- [18] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. CIFAR-10 and CIFAR-100 datasets. 5, 1
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. *Commun. ACM*, 60(6):84–90, 2017. 1
- [20] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*, pages 9992–10002. IEEE, 2021. 2
- [21] Niki Parmar, Prajit Ramachandran, Ashish Vaswani, Irwan Bello, Anselm Levskaya, and Jonathon Shlens. Stand-alone self-attention in vision models. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, Decem-*

- ber 8-14, 2019, Vancouver, BC, Canada, pages 68–80, 2019. 2
- [22] Zhiliang Peng, Wei Huang, Shanzhi Gu, Lingxi Xie, Yaowei Wang, Jianbin Jiao, and Qixiang Ye. Conformer: Local features coupling global representations for visual recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 367–376, 2021. 4
 - [23] Tobias Plötz and Stefan Roth. Neural nearest neighbors networks. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 1095–1106, 2018. 2
 - [24] PyTorch Developers. *PyTorch: Open Source Machine Learning Framework*. PyTorch Foundation, 2025. Version 2.9. Available at <https://pytorch.org/>. 4
 - [25] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. 2
 - [26] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1874–1883, 2016. 1, 2
 - [27] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. 1, 5
 - [28] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. 4
 - [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 2, 3, 7, 8
 - [30] Pichao Wang, Xue Wang, Fan Wang, Ming Lin, Shuning Chang, Hao Li, and Rong Jin. KVT: k-nn attention for boosting vision transformers. In *Computer Vision - ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXIV*, pages 285–302. Springer, 2022. 2, 5, 7, 8
 - [31] Xiaolong Wang, Ross B. Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*, pages 7794–7803. Computer Vision Foundation / IEEE Computer Society, 2018. 2
 - [32] Kun Yuan, Shaopeng Guo, Ziwei Liu, Aojun Zhou, Fengwei Yu, and Wei Wu. Incorporating convolution designs into visual transformers. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*, pages 559–568. IEEE, 2021. 2

Attention Via Convolutional Nearest Neighbors

Supplementary Material

10. Dataset and Preprocessing Details

We conduct experiments on CIFAR-10 and CIFAR-100 [18], two standard benchmarks widely used for evaluating architectures in computer vision.

CIFAR-10 consists of 60,000 color images (32×32 resolution) across 10 object categories, with 50,000 training images and 10,000 test images. CIFAR-100 contains 60,000 color images (32×32 resolution) across 100 classes with the same train-test split. Both datasets use balanced class distribution, with 5,000 training samples per class in CIFAR-10 and 500 training samples per class in CIFAR-100.

To accommodate the two architectures (CNNs and Vision Transformers), we apply two different preprocessing strategies. For VGG-11 experiments, we maintain the original 32×32 image resolution. This preserves the inherent spatial locality that CNNs exploit, ensuring fair comparison between convolutional and ConvNN operations. At this resolution, ConvNN’s neighbor selection mechanism operate on relatively dense feature spaces, allowing us to isolate the benefits of hybrid spatial-feature selection. For ViT-Tiny experiments, we resize input images to 224×224 resolution to align with Vision Transformer’s patchification mechanism. ViT divides images into 16×16 patches, requiring larger resolution inputs for meaningful patch-level feature extraction. This resolution allows for proper positional encoding and attention mechanisms designed for the transformer architecture.

For all experiments, we apply per-channel normalization using CIFAR dataset statistics. CIFAR-10’s mean and standard deviation is $[0.4914, 0.4822, 0.4465]$, $[0.2470, 0.2435, 0.2616]$ and CIFAR-100’s mean and standard deviation is $[0.5071, 0.4867, 0.4408]$, $[0.2675, 0.2565, 0.2761]$. Normalization is applied after resizing (where applicable) and before model training.

Notably, we apply no data augmentation techniques (e.g., random cropping, horizontal flips, color jittering, or cutout) to ensure fair and controlled comparisons across all methods. We acknowledge that production models would benefit from data augmentation and plan to investigate how ConvNN perform with different augmentation strategies and whether their regularization benefits complement or interfere with augmentation.

11. Hyperparameters

The main hyperparameters are described in Sec. 6 of the main paper. Below we document additional hyperparameters used in supplementary ablation studies.

VGG-11 Dropout rate of 0.5 is applied after fully connected

layers for all VGG-11 models. Standard Conv2D baseline uses 3×3 kernels. Default Conv2D neighbor count is $k = 9$, ablated across $k \in [1, 2, \dots, 12]$ in Figure 4.

ViT-Tiny Standard Vision Transformer with 12 layers, 192 hidden dimensions, 768 MLP dimensions, and single attention head (as in main paper). For ConvNN-Attention variants, we employ 1D depthwise convolution during weighted aggregation with weights initialized uniformly to 1.0 and no bias term. Kernel size and stride both equal the neighbor count k . Default neighbor count is $k = 9$, ablated across $k \in [2, 3, \dots, 12]$ in Figure 7.

Attention Baselines Table 2 compares the following baseline attention variants with the configurations below:

- Sparse Attention: context window of 49 with local mode enabled
- Local Attention: window size of 128
- KVT: default $k = 9$, ablated across $k \in [2, 3, \dots, 12, 100]$ in Figure 7

All attention variants use linear projections with no bias term to ensure fair comparison with ConvNN.

Hybrid Branching Hybrid branching’s mixing parameter $\lambda \in [0.000, 0.125, 0.250, \dots, 1.000]$ controls the proportion of channels allocated to convolutional and ConvNN branches. Convolutional branch uses kernel size of 3×3 and ConvNN with neighbor count $k = 9$ to maintain comparable parameter budgets (Table 1 and 3). $\lambda = 0$ represents pure Conv2D; $\lambda = 1$ represents pure ConvNN.

Sparse Neighbor Search VGG-11 uses random and spatial selection strategies, ablated across $r \in [4, 6, 8, \dots, 30]$ in Figure 5. ViT-Tiny uses default $r = 32$, ablated across $r \in [16, 32, 48, \dots, 192]$ in Figure 8.

Positional Encoding In Table 1, when positional encoding is applied, two channels encoding horizontal and vertical coordinates (i, j) are concatenated to input features before similarity computation. Optional application before weighted aggregation is explored in Table 3.

Patch-Level Comparison Optional PixelUnshuffle[26] pre-processing with patch size $p \in [1, 2]$ increases channels by p^2 while reducing spatial resolution by p , enabling region-aware feature comparisons before neighbor selection (Table 4).

12. Hybrid Branching in VGG-11

To systematically evaluate the hybrid branching layer, we vary the mixing parameter $\lambda \in [0, 1]$ to interpolate between pure convolution (at $\lambda = 0$) and pure ConvNN (at $\lambda = 1$) configurations. We explore two configurations to explore the benefits of positional encoding: (1) positional encoding in similarity computation, and (2) positional encoding in both similarity computation and weighted aggregation.

Table 3. Ablation study on hybrid branching parameter λ for VGG-11 on CIFAR-10 and CIFAR-100 with positional encoding used in similarity computation and weighted aggregation. Bold values indicate performance improvements over the Conv2D baseline.

CIFAR-10				
Layer Type	Test Loss	Acc (%)	GFLOPS	# Params
Conv2D	1.529	80.65	0.293	130.015M
λ	0.125	1.393	81.34	0.331 130.021M
	0.250	1.265	81.51	0.331 130.028M
	0.375	1.522	80.72	0.331 130.034M
	0.500	1.218	82.34	0.331 130.040M
	0.625	1.244	81.58	0.331 130.046M
	0.750	1.232	81.06	0.331 130.052M
	0.875	1.248	78.04	0.331 130.059M
ConvNN	1.916	63.75	0.331	130.065M
CIFAR-100				
Layer Type	Test Loss	Acc (%)	GFLOPS	# Params
Conv2D	3.939	44.83	0.293	130.384M
λ	0.125	3.598	46.39	0.331 130.390M
	0.250	3.641	47.52	0.331 130.396M
	0.375	3.461	48.78	0.331 130.402M
	0.500	3.417	49.65	0.331 130.409M
	0.625	3.525	49.12	0.331 130.415M
	0.750	3.274	48.53	0.331 130.421M
	0.875	3.624	45.69	0.331 130.427M
ConvNN	3.977	37.38	0.331	130.433M

Positional Encoding in Similarity Only When positional encoding is applied only to similarity computation, projections f_K and f_Q include both feature and spatial dimensions, enabling spatially-aware neighbor selection. However, the value projection f_V includes features only, keeping the parameter count unchanged compared to the feature-only baseline. Results shown in Table 1.

Positional Encoding in Both Similarity and Aggregation

When positional encoding is incorporated into both similarity computation and weighted aggregation, all projections f_K , f_Q , and f_V include both feature and spatial dimensions. The parameter count increase modestly to account for the additional spatial dimensions from the value projection f_V . Results shown in Table 3.

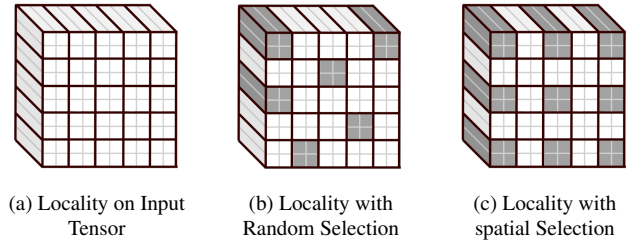


Figure 9. Patch-level locality with neighbor candidate selection.

Across both configurations, adding positional encoding consistently improves accuracy while maintaining relative losses. However, the improvement from including positional encoding in the weighted aggregation step are marginal compared to including positional encoding in similarity computation. This suggests that spatial awareness is more beneficial during neighbor selection rather than during the final aggregation, indicating that learned feature-based aggregation is more effective than spatially-informed weighting.

13. Patch-Level Comparison

Individual feature comparisons may fail to capture local texture and structural patterns. To address this limitation, ConvNN supports patch-level comparisons of size $p \times p$, where $p = 1$ corresponds to the standard per-feature case. This is implemented by introducing a PixelUnshuffle [26] preprocessing layer, which increases the number of channels by a factor of p^2 while reducing spatial resolution by p . This transformation enables richer, region-aware feature interactions prior to neighbor selection. Formally, for an input of shape $[c, w, h]$, its patchified version has shape $[cp^2, w/p, h/p]$. Figures 9b and 9c illustrate the patchification process in the context of random and spatial feature selection, respectively.

Table 4 reveals an important trade-off. Increasing patch size p from 1 to 2 reduces computational costs (GFLOPS) but significantly degrades performance across all metrics. Notably, the hybrid branching ($\lambda = 0.5$) shows better robustness to patch size variation compared to pure ConvNN.

These results highlight the importance of spatial resolution in our method. The parameter count increases substantially from increasing patch size, specifically due to scaling with p^2 in the number of channels. For $p = 2$ in ConvNN, this results in doubling of parameters (from 130M to 268M). This parameter increase in both hybrid branching and ConvNN likely explains the degraded performance, as the models may require more training to converge properly. Furthermore, the poor performance on small images of CIFAR-10 and CIFAR-100 suggests that maintaining fine-grained spatial information ($p = 1$) is crucial for hybrid branching and ConvNN, and the computational efficiency from increased patch size comes at a significant performance cost.

Table 4. Patch-Level Comparisons with $p \in [1, 2]$ for VGG-11 on CIFAR-10 and CIFAR-100 datasets with no sparse neighbor search.

CIFAR-10					
Layer Type	p	Test Loss	Acc (%)	GFLOPS	# Params
CONV2D	1	1.529	80.65	0.293	130.015M
Branching ($\lambda = 0.5$)	1	1.424	81.26	0.325	130.015M
Branching ($\lambda = 0.5$)	2	1.403	74.81	0.307	199.152M
CONVNN	1	2.694	54.00	0.325	130.015M
CONVNN	2	1.631	60.32	0.307	268.289M
CIFAR-100					
Layer Type	p	Test Loss	Acc (%)	GFLOPS	# Params
CONV2D	1	3.939	44.83	0.293	130.384M
Branching ($\lambda = 0.5$)	1	3.474	48.37	0.325	130.384M
Branching ($\lambda = 0.5$)	2	4.242	37.01	0.308	199.512M
CONVNN	1	5.011	29.50	0.325	130.384M
CONVNN	2	4.487	28.17	0.308	268.658M

14. Convolution Aggregation Variation in CONVNN-Attention for ViT

We explore three distinct convolutional operations in the weighted aggregation phase of the CONVNN-Attention for ViT: Depthwise (D), Depthwise-Separable (DS) [7], and Regular (R) convolutions. Depthwise convolution maintains channel separation with weights initialized to 1.0. Depthwise-Separable convolution reduce parameters by factorizing spatial and channel operations. Regular convolution apply fully learnable parameters across all channels. All convolutional variations consider all candidate neighbors without sparse selection.

Table 5 reveals important insights. Depthwise convolution achieves the best accuracy with minimal parameter overhead, maintaining parameter counts close to the baseline attention model. Regular convolutions significantly increase parameters ($\approx 5.5M \rightarrow \approx 9.5M$), indicating that fully learnable convolutions are inefficient for this aggregation task. Depthwise-Separable convolution unexpectedly underperforms Depthwise, suggesting that inter-channel mixing through spatial-channel factorization does not improve learning in this context. These results indicate that maintaining channel separation is beneficial for effective aggregation.

Importantly, all three convolution variations produce comparable test loss values (ranging from 1.939 to 1.978 on CIFAR-10), suggesting that the weighted aggregation is less critical than the neighbor selection mechanism itself. This reinforces our key finding that CONVNN’s improvement stem primarily from flexible neighbor selection rather than the specific aggregation strategy.

Table 5. Convolution variations comparison in CONVNN-Attention for ViT-Tiny. D, DS, and R denote Depthwise, Depthwise-Separable, and Regular convolutions, respectively. Depthwise convolutions are initialized with unit weights. All CONVNN variations use full candidate sampling (no sparsification) and $k = 9$.

CIFAR-10					
Layer Type	Test Loss	Acc (%)	GFLOPS	# Params	
Attention [29]	1.558	77.41	2.507	5.479M	
CONVNN (D)	1.978	73.67	2.333	5.500M	
CONVNN (DS)	1.939	71.93	2.333	5.942M	
CONVNN (R)	1.940	72.90	2.333	9.460M	
CIFAR-100					
Layer Type	Test Loss	Acc (%)	GFLOPS	# Params	
Attention [29]	3.817	50.07	2.507	5.496M	
CONVNN (D)	3.983	48.67	2.333	5.517M	
CONVNN (DS)	4.140	43.75	2.333	5.959M	
CONVNN (R)	3.900	47.17	2.333	9.477M	

15. Learning Rate Sensitivity Analysis

While the main paper employs a learning rate of 10^{-4} for VGG-11 training, we investigate the robustness of the hybrid branching and fully CONVNN layers across different learning rates. This ablation examines three different learning rates of 10^{-3} , 10^{-4} , and 10^{-5} .

Figure 10 demonstrates that the hybrid branching approach remains stable and effective across all learning rate settings on both CIFAR-10 and CIFAR-100. Notably, pure CONVNN outperforms the CONV2D baseline at both extreme learning rates (10^{-3} and 10^{-5}), suggesting it benefits from either aggressive or conservative optimization strategies. However, the hybrid branching consistently achieves the best performance across all learning rates, confirming that its regularization benefits are robust to optimization hyperparameters.

16. Included Code

We provide sample code with the supplemental material to enable reproducibility. Algorithm 1 presents the core CONVNN operation in simplified pseudocode for clarity. The actual implementations used in experiments are more complex due to practical considerations: they handle batched tensor operations, implement sparse neighbor selection operations, and manage positional encoding concatenation. Despite these implementation details, the fundamental three-step procedure—similarity computation, neighbor selection, and weighted aggregation—remains unchanged. A simplified reference implementation (simplified_convnn.py) is also provided. Both VGG-11 and ViT-Tiny experiments are fully reproducible using the provided code.

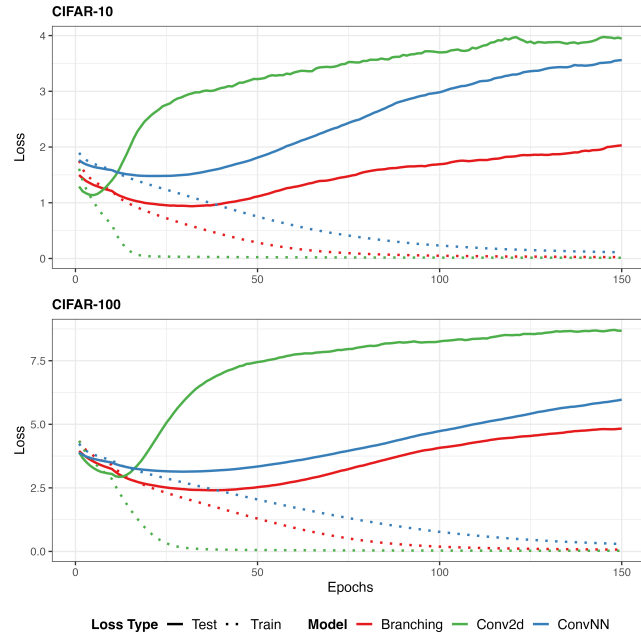
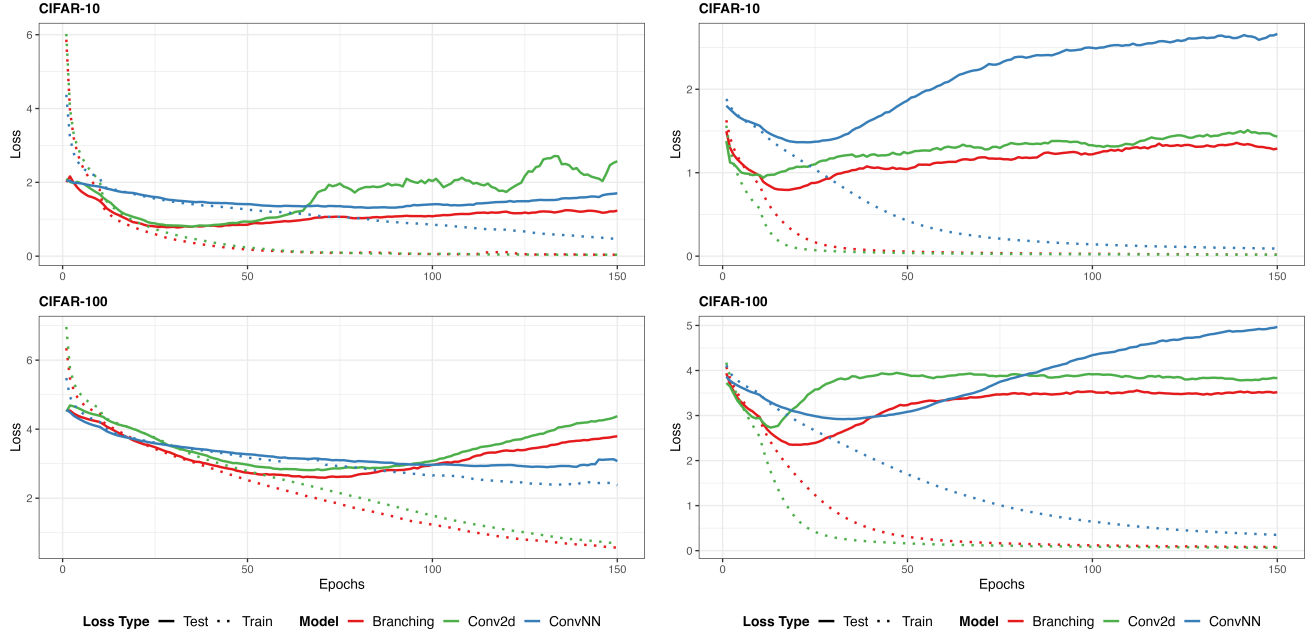


Figure 10. Learning rate sensitivity analysis for Branching and ConvNN on VGG-11. For visualization purposes, all loss curves are smoothed using a moving average with a window size of 10.